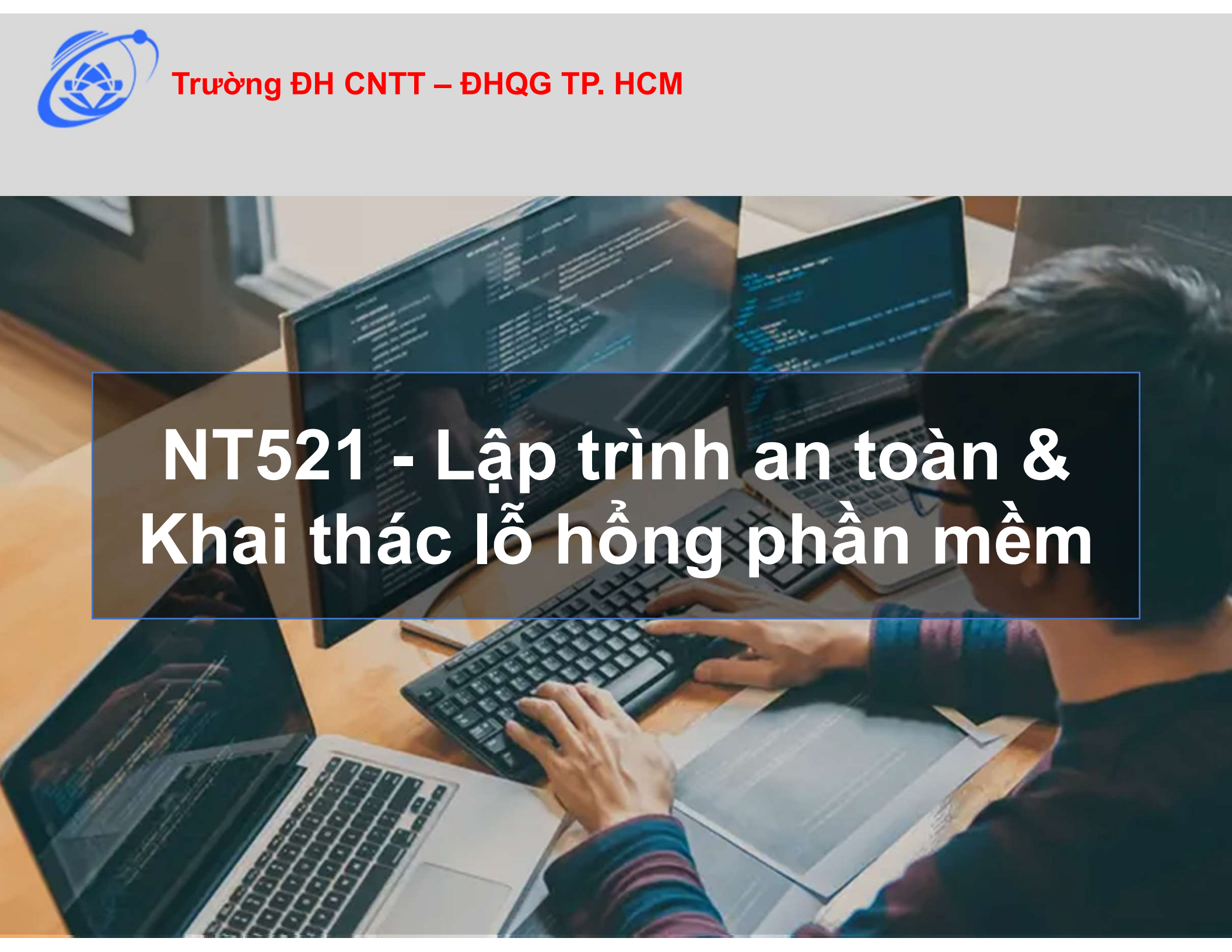




Trường ĐH CNTT – ĐHQG TP. HCM



NT521 - Lập trình an toàn & Khai thác lỗ hổng phần mềm

Bài tập Buffer overflow &Tràn số nguyên



- Bài tập Buffer overflow
- Một số trường hợp CVE thực tế
- Lỗi hỏng Tràn số nguyên (Integer Overflow)

Cho biến **char buf[50]**. Cách gọi hàm nào dưới đây có thể gây ra lỗi hỏng buffer overflow?

- A. gets(buf) ✓
- B. fgets(buf, 50, stdin)
- C. scanf("%s", buf) ✓
- D. Tất cả đều gây ra lỗi buffer overflow

Cho các hàm như bên dưới với **main** → **test** → **done**.
Cho biết lỗ hổng buffer overflow xảy ra có thể ghi đè các giá trị đang nằm ở đâu?

- A. Stack frame hàm gets
- B. Stack frame hàm test ✓
- C. Stack frame hàm done
- D. Stack frame cả 3 hàm

```
int done() {  
    printf("Not thing...")  
}  
  
int test()  
{  
    char buf[25];  
    gets(buf);  
    done();  
}  
  
int main()  
{  
    printf("Hello");  
    test();  
}
```

Cho 1 file có sử dụng hàm **gets** và được biên dịch với option **-fno-stack-protector** và **-z noexecute**, nhận định nào đúng?

- A. File này không thể bị khai thác buffer overflow.
- B. File này có thể bị khai thác để ghi đè 1 số biến cục bộ trong stack frame của hàm có lỗi hổng. ✓
- C. File này có thể bị khai thác để truyền và thực thi shellcode do người dùng tự định nghĩa.
- D. File này có thể bị khai thác ở cả 2 mức độ.

Khi khai thác lỗ hổng buffer overflow, bạn đã dùng phương pháp nào để xác định độ dài chuỗi input cần nhập?

- A. Đọc code assembly tìm khoảng cách giữa chuỗi input và chỗ cần ghi đè.
- B. Debug để tìm địa chỉ cụ thể của chuỗi input và chỗ cần ghi đè, sau đó tính khoảng cách.
- C. Em tăng dần độ dài đến khi nào lỗi thì thôi 😊
- D. Tất cả là nhờ may mắn :D

Bài tập 1 – buffer1

```
#include <stdio.h>
#include <stdlib.h>
void get_shell(){
    printf("Yay! You've called get_shell function\n");
    system("/bin/sh");
    exit(0);
}
void buf_overflow(){
    int a = 0;
    char buf[25];
    gets(buf);
    puts(buf);
    if (a == 0xdeadbeef){
        printf("Return to my caller...\n");
        return;
    }
    else exit(0);
}
int main(){
    buf_overflow();
}
```

Yêu cầu: khai thác hàm **buf_overflow()** để gọi được hàm **get_shell()** và truyền lệnh

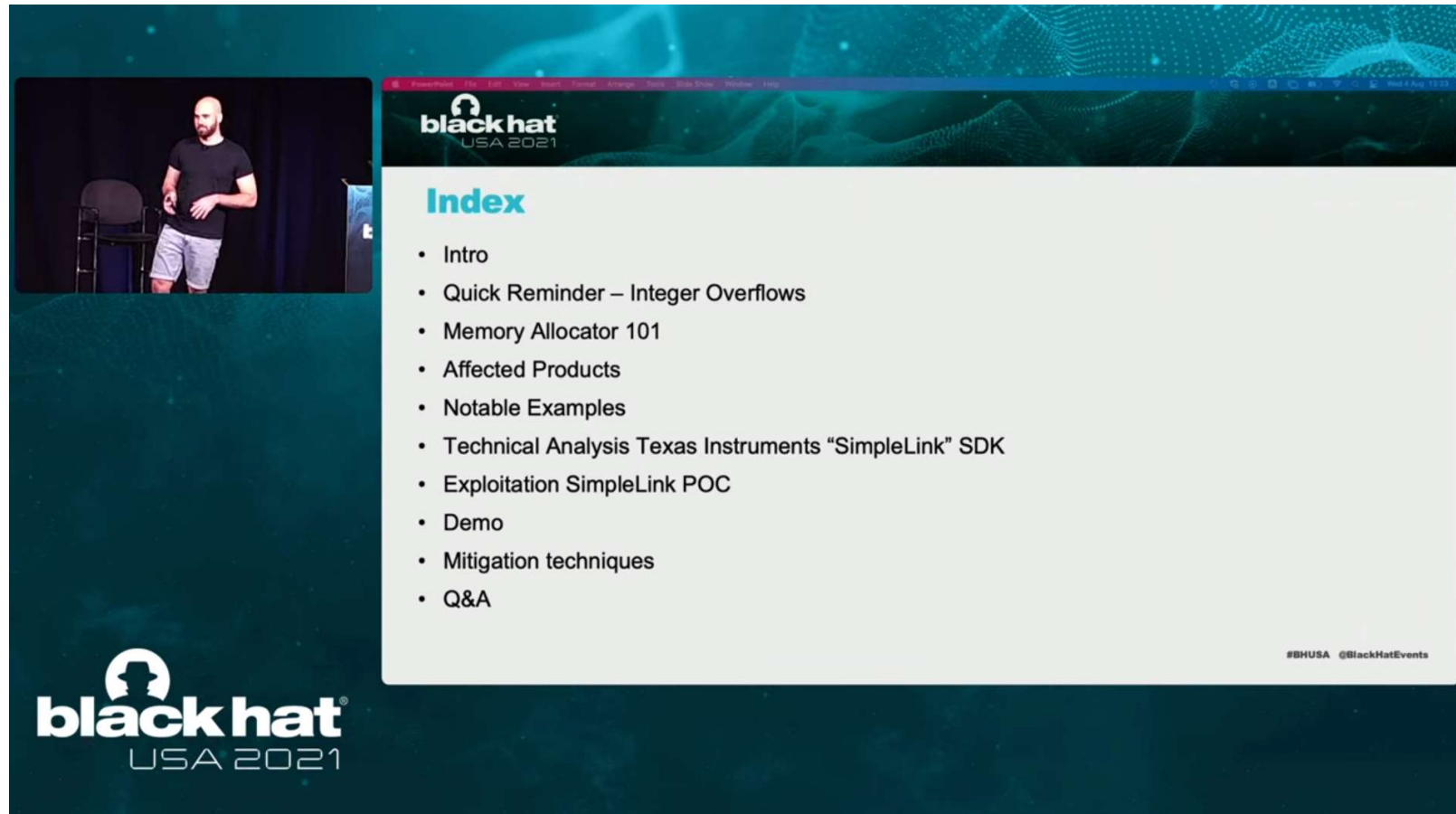


ERROR: BadAlloc! - Broken Memory Allocators Led to Millions of Vulnerable IoT & Embedded Devices:

- Link: <https://www.blackhat.com/us-21/briefings/schedule/#error-badalloc---broken-memory-allocators-led-to-millions-of-vulnerable-iot-and-embedded-devices-23135>
- Youtube: <https://www.youtube.com/watch?v=ISvygMc8uc0>

Khai thác thực tế

Integer Overflow



ERROR: BadAlloc! - Broken Memory Allocators Led to Millions of Vulnerable IoT & Embedded Devices:

- Link: <https://www.blackhat.com/us-21/briefings/schedule/#error-badalloc---broken-memory-allocators-led-to-millions-of-vulnerable-iot-and-embedded-devices-23135>
- Youtube: <https://www.youtube.com/watch?v=ISvygMc8uc0>



Lỗi hỏng tràn số nguyên

Integer Overflow Vulnerability

- Trong lập trình máy tính, Integer Overflow xảy ra khi một phép toán số học cố gắng tạo **một giá trị số nằm ngoài phạm vi có thể được biểu diễn bằng một số chữ số nhất định** - cao hơn giá trị lớn nhất hoặc thấp hơn giá trị nhỏ nhất có thể biểu diễn.

⇒ Integer Overflow hiểu đơn giản là lỗi hỏng trong việc máy tính lưu trữ và tính toán các con số gây ra sai lệch và khiến cho chương trình chạy sai với mong muốn của lập trình viên

** Một số định nghĩa bao gồm luôn cả sự sai số trong lưu trữ số thực*

Address Memory RAM

154

153

152

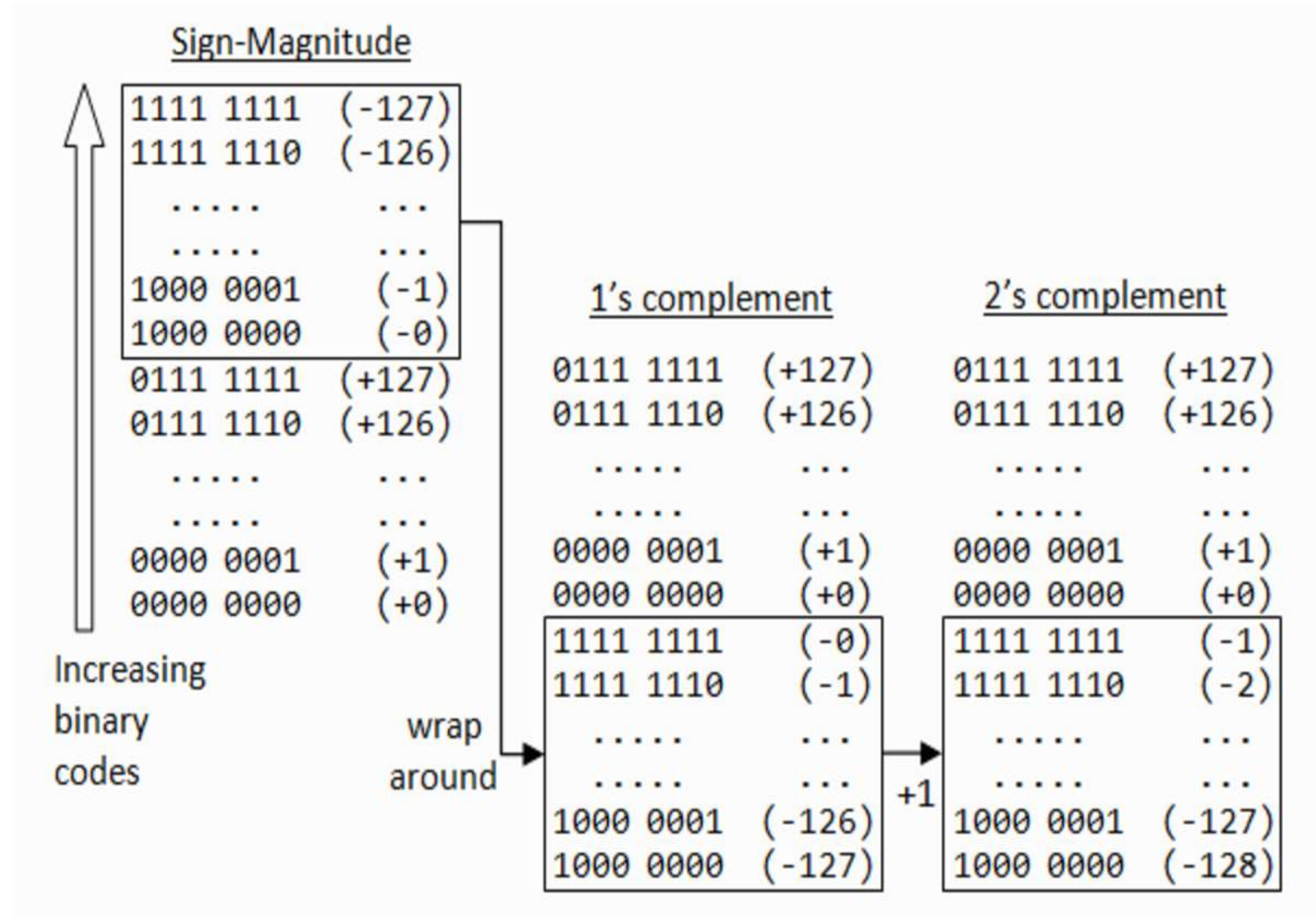
151

150

149

The diagram shows a vertical stack of six memory cells. Each cell is represented by a rectangle. To the left of each rectangle is its address: 154, 153, 152, 151, 150, and 149 from top to bottom. A blue line connects each address to its corresponding rectangle. Above the stack, the word 'Memory' is written, and below the stack, the word 'RAM' is written. A large upward-pointing arrow is positioned above the stack, and a large downward-pointing arrow is positioned below the stack.

- Số âm:



Integer Overflow: Các kiểu dữ liệu trong C



Data type	Size(bytes)	Range	Format string
Char	1	-128 to 127	%c
Unsigned char	1	0 to 255	%c
Short or int	2	-32,768 to 32,767	%i or %d
Unsigned int	2	0 to 65535	%u
Long	4	-2147483648 to 2147483647	%ld
Unsigned long	4	0 to 4294967295	%lu
Float	4	3.4 e-38 to 3.4 e+38	%f or %g
Double	8	1.7 e-308 to 1.7 e+308	%lf
Long Double	10	3.4 e-4932 to 1.1 e+4932	%Lf

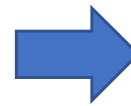


Lỗi hỏng Trần số nguyên

Integer Overflow: Ví dụ



```
C++ test.cpp X
C: > Document > LapTrinhAnToanVaKhaiThacLoHongPhanMem > Semina
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int a = 4278190080;
7      cout << "a = " << a << '\n';
8      unsigned int b = 4299360564;
9      cout << "b = " << b << '\n';
10     cout << "a - b = " << a - b << '\n';
11     int c = 1;
12     unsigned int d = 4294967295;
13     cout << "c - d = " << c - d << '\n';
14     return 0;
15 }
16
```



```
C:\Document\LapTrinhAnT
a = -16777216
b = 4393268
a - b = 4273796812
c - d = 2
```



Lỗ hổng Trần số nguyên

Xác định giá trị tràn số?



- Trong hệ biểu diễn **4 bit**
 - Không dấu: 0 đến 15
 - Có dấu: -8 đến 7

Giá trị tràn số của **x** trong hệ **n** bit:

- Hệ không dấu: $x \bmod 2^n$
- Hệ có dấu: $x \bmod 2^{n-1}$, nếu $x/2^{n-1}$ ra kết quả lẻ thì bit dấu là 1, kết quả chẵn thì bit dấu là 0.

Giả sử cho số **50**, trong hệ trên sẽ có giá trị bao nhiêu?

- Không dấu?
- Có dấu?

Giả sử cho kết quả **10 + 19**, trong hệ trên sẽ có giá trị bao nhiêu?

- Không dấu?
- Có dấu?

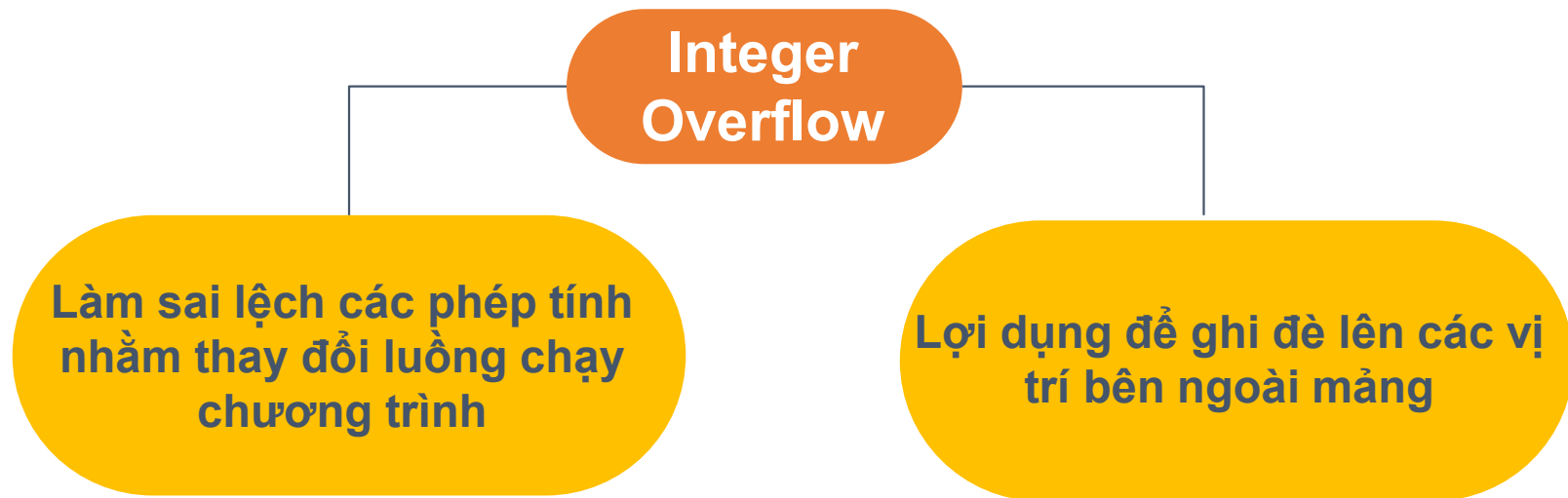


Lỗi hỏng Trần số nguyên

Integer Overflow: Cách khai thác



Khai thác lỗi hỏng này như thế nào?



Integer overflow: Ví dụ 1

- Nhập **b** bằng bao nhiêu?

```
#include <iostream>
using namespace std;

int main(){
    int a = 0x98; //152
    int b = ?;
    printf("%d\n", a+b);
    if (b < 0 || a + b > 0)
        printf("Try again");
    else
        printf("Ok. You've passed");
}
```

Integer overflow: Ví dụ 2

```
int * myfunction(int * array, int len) {
    int * myarray, i;

    myarray = malloc(len * sizeof(int));
    if (myarray == NULL) {
        return -1;
    }

    for (i = 0; i < len; i++)
        myarray[i] = array[i];

    return myarray;
}
```

malloc nhận tham số là **kích thước** (tính theo byte) cần cấp (không dấu). **len*sizeof(int)** có thể overflow.

- Nếu **len** đủ lớn để **len*sizeof(int)** tràn thành 0 → malloc cấp vùng nhớ 0 byte cho array nhưng vòng lặp for vẫn ghi đè dữ liệu vào vùng nhớ tại đó.
- Nếu tràn số để malloc cấp ít vùng nhớ hơn mình cần → vòng lặp for vẫn có thể ghi đè ra bên ngoài mảng

Integer overflow: Ví dụ 3 (picoCTF)



Yêu cầu: Mua 1 flag có giá **100,000 coins** với tài khoản chỉ có **1,100 coins**.

Có 2 dạng flag có thể mua, sau khi mua thì tiền trong tài khoản sẽ cập nhật.

1. Flag chính giá 100,000 coins.
2. Flag phụ giá 900 coins cho mỗi flag

```
int account_balance = 1100;
int number_flags = 0;
int total_cost = 0;

printf("1337 flags cost 100000 dollars, and we only have 1 in stock\n");
printf("Enter 1 to buy one");

int bid = 0;
fflush(stdin);
scanf("%d", &bid);

if(bid == 1){

    if(account_balance > 100000){
        FILE *f = fopen("flag.txt", "r");
        if(f == NULL){

            printf("flag not found: please run this on the server\n");
            exit(0);
        }
        char buf[64];
        fgets(buf, 63, f);
        printf("YOUR FLAG IS: %s\n", buf);
    }

    else{
        printf("\nNot enough funds for transaction\n\n\n");
    }
}}
```



Integer overflow: Ví dụ 3 (picoCTF)



```
if(auction_choice == 1){
    printf("These knockoff Flags cost 900 each, enter desired quantity\n");

    int number_flags = 0;
    fflush(stdin);
    scanf("%d", &number_flags);
    if(number_flags > 0){
        int total_cost = 0;
        total_cost = 900*number_flags;
        printf("\nThe final cost is: %d\n", total_cost);
        if(total_cost <= account_balance){
            account_balance = account_balance - total_cost;
            printf("\nYour current balance after transaction: %d\n\n", account_balance);
        }
        else{
            printf("Not enough funds to complete purchase\n");
        }
    }
}
```

Code cập nhật account_balance sau khi mua flag phụ → integer overflow
→ number_flags đủ lớn → total_cost có thể tràn thành âm → tăng account_balance
→ Cần nhập number_flags là bao nhiêu?



- Cấp phát kiểu dữ liệu đủ lớn cho giá trị lớn nhất và nhỏ nhất để lưu được tất cả các giá trị có thể tính ra.
- Sắp xếp các phép toán cẩn thận, kiểm tra giá trị các biến
- Cẩn thận khi ép kiểu
- Đánh giá, kiểm thử mã nguồn, ứng dụng
- Lựa chọn ngôn ngữ hạn chế tràn số nguyên như Python hoặc dùng các thư viện IntegerLib hoặc SafeInt cho C/C++